Kava Chess Clock — Attendance Forecasting for Kava Social Chess Club

Harold Gonzalez · Distributed Systems for Data Science · NCF · Spring 2026

Live app: https://kava-social-attendance-predictor-cwlaxs6ygbxud7z48884zq.streamlit.app/ · **Repo:** github.com/Ruptzy/kava-social-attendance-predictor

What it predicts

Unique-player attendance for a future Kava Social chess bracket night in Bradenton, FL. Two heads share one feature row: a regressor for `attendance_count` and a classifier for `high_turnout` (1 if attendance ≥ historical median, 15 players). The tool is built for the people who run the bracket — to plan boards, clocks, and staffing — not to forecast game outcomes, openings, or player performance.

Data source

72 historical Kava Social bracket nights (Sep 2022 – Apr 2026), stored as game-level rows (`Date, Time, White, Black, Winner`). One bracket night contains 20–50 such rows; I aggregate to event level — attendance for a date is the count of unique non-null players in either color column. Weather is pulled from the free Open-Meteo archive + forecast APIs for Bradenton (27.50 N, –82.57 W).

Pipeline architecture

raw bracket logs → S3 bronze → PySpark silver/gold (S3) → scikit-learn + MLflow → Streamlit Cloud

Two distributed/cloud stages: (1) bronze ingestion to **AWS S3** (`s3://kava-chess-pipeline-352435704328/bronze/`) via boto3; (2) **PySpark** transformation reading raw TSV from S3, applying name normalization + deduplication + window-function lag/rolling features, writing **silver** (game-level Parquet) and **gold** (event-level Parquet) back to S3. A pandas mirror runs locally for fast iteration with the same schema. Models are trained with scikit-learn, tracked in **MLflow**, and persisted to `models/`. The Streamlit app on **Streamlit Community Cloud** loads the model directly.

Model approach

A **four-way bake-off** is run against a chronological holdout (the most recent 14 events, never seen during training): a **Naive baseline** (always predict last night's attendance), **Ridge Regression**, **Random Forest**, and **Gradient Boosting**. Naive: MAE 3.43; Ridge: 4.71; **Random Forest: 2.30 (selected)**; Gradient Boosting: 2.56. Random Forest beats the baseline by ~1.1 players on the unseen holdout and is the model the live app serves.

Features (~40 total) come from five plain-language families exposed in the UI tabs: **recent attendance momentum** (`previous_event_attendance`, rolling 3/5/10-event averages, trend deltas), **calendar timing** (month, day-of-week, week-of-year, days-since-last-event, holiday-week/school-break flags), **Bradenton weather** (daytime high/low/mean temperature, feels-like, daily and 7–11 PM event-window precipitation in millimetres, 7–11 PM humidity, daily peak humidity, wind, thunderstorm flag, plus a 0–5 weather-discomfort score from humidity / heat / rain / wind / storm), and **prior-event community momentum** (previous unique players, new vs returning split, games per player, rolling-3 averages of each). Florida rain is too noisy to use as a binary yes/no feature, so the headline weather signal is the comfort score; precipitation is kept as a continuous millimetre amount. **No same-night features** are used as predictors — they would not be observable when planning the night. Evaluation is the chronological holdout plus 4-fold **time-series cross-validation** on the training portion.

Classifier (high vs low turnout): Random Forest, 4-fold time-series-CV **accuracy ~ 64%, F1 ~ 0.59**. All four candidate regression runs and the classifier are logged in MLflow.

What I learned

Three things bit harder than expected. **First, deduplication and name normalization** — the same player appears as "Gonzalez, Harold" / "Harold", "Cruz, Omar" / "Omar", a handful of variants. I kept the merge list conservative (only confirmed mappings) so attendance counts don't get inflated by aggressive matching. **Second, leakage was easy to introduce** — every interesting same-night number (game count, draw rate, new-player count) is information you only have *after* people show up. The rule of "predict event E using only events strictly before E" forced me to use `shift(1)` everywhere and verify with TimeSeriesSplit instead of regular CV. **Third, plain language beats jargon** — my first UI revision had "CV MAE 3.67" and "■ regressor estimate" labels everywhere; the redesign replaces them with "usually off by about 2.3 players" and "predicted attendance," which made the dashboard feel like a real planning tool. If I rebuilt

it I would (a) add an automated weekly retraining job (GitHub Actions + S3), (b) wire a real FastAPI endpoint so the test script could hit a REST API, and (c) collect "did the event actually happen on Sunday or get moved?" as a feature, since holiday-week reschedules visibly distort attendance.